GWV-xuk Backend — A Complete Beginner's Guide

> A file-by-file tour of the FastAPI e-commerce backend, the security we added, and everything you still need to build the full project (storefront, admin dashboard, WhatsApp triage bot).

Written for a junior developer who wants to understand the **flow** of the whole backend, not just memorise it.

Table of contents

1. The big picture (with an analogy)
2. The stack, explained like you're new
3. How one request travels through the app
4. Running the project locally
5. The project tree
6. File-by-file walkthrough
7. Security hardening — the 4 fixes explained
8. How the M-Pesa payment flow works
9. Every endpoint that exists today
10. Admin dashboard — endpoints to build
11. Leads + WhatsApp triage bot — design & endpoints
12. Recommended backend folder structure
13. Files needed to complete the full project
14. Database & Row Level Security
15. Prioritised next steps
16. Glossary for juniors

1. The big picture (with an analogy)

Think of the backend as a **restaurant**.

Restaurant thing	Backend equivalent
The dining room (where customers sit)	Your **React website / mobile app** (the frontend)
The **waiter** who takes orders	Your **API routes** (FastAPI `@router.get/post/...`)
The **bouncer** at the door checking IDs	**Authentication + authorization** (`get_current_user`, `require_admin`)
The **kitchen** that cooks	Your **business logic** (`services/`)
The **pantry/fridge** where ingredients live	**Supabase** (database + file storage)
The **cashier / card machine**	**M-Pesa (Daraja)** payments
A **messenger** who calls you when your food is ready	**Webhooks** (M-Pesa calls *us* back)

The frontend never touches the database directly. It always asks the waiter (the API). The waiter checks your ID, writes down the order, the kitchen cooks, and the pantry stores the ingredients. That separation is what keeps a system understandable and safe.

Golden rule to remember forever:

> The frontend is never trusted. It can be edited by anyone with browser dev-tools. All truth (prices, who you are, stock) must be decided by the backend.

2. The stack, explained like you're new

- **Python** — the language we write the backend in.
- **FastAPI** — a web framework. You write small Python functions; FastAPI turns them into URLs (`/products`, `/orders/checkout`). It also auto-generates docs at `/docs`.
- **Uvicorn** — the server that actually runs FastAPI. `uvicorn app.main:app` means "run the object called `app` inside `app/main.py`".
- **Pydantic** — validates and shapes data. A class like `CheckoutRequest` describes exactly what a JSON request must look like; FastAPI rejects bad requests automatically.
- **Supabase** — a hosted PostgreSQL database with extras: **Auth** (login / JWTs), **Storage** (files/images), and a REST API. We use the Python `supabase` client to talk to it.
- **JWT (JSON Web Token)** — a signed "wristband" the server gives you when you log in. You show it on every request. It proves who you are without sending your password again.
- **M-Pesa Daraja** — Safaricom's API. We ask it to pop a payment prompt (STK push) on the customer's phone, and it calls us back when payment finishes.
- **Webhook** — a reverse API call: instead of us calling them, *they* call our URL with the result.
- **Resend** — a service that sends emails (we use it for receipts).
- **OpenRouter** — an LLM gateway; we use it for the AI shopping assistant (and later will reuse it for the WhatsApp bot).

Senior advice: Learn the difference between *synchronous* (one thing at a time) and *asynchronous* (`async def` — can handle other work while waiting on the network). FastAPI is async; that's why database/HTTP calls are the slow part and why blocking calls

hurt performance later.

3. How one request travels through the app

Example: the frontend calls `POST /orders/checkout` to buy a phone.

```
[React app]
    |  (1) HTTP POST /orders/checkout  +  Authorization: Bearer <JWT>  + JSON body
    v
[FastAPI app object in app/main.py]
    |  (2) CORS check (is this browser origin allowed?)
    |  (3) Route matching: find the function decorated with @router.post("/checkout'
    v
[app/routers/orders.py :: process_checkout]
    |  (4) Pydantic validates the JSON body against CheckoutRequest
    |  (5) (If a route needs it) dependencies run: get_current_user / require_admin
    |  (6) Business logic: read DB prices, validate stock, compute total
    |  (7) Supabase: INSERT order, UPDATE product stock
    |  (8) Daraja: send STK push to the phone
    v
[FastAPI serialises the return value to JSON]
    v
[React app receives {status: "success", order_id: "GWV-1234", ...}]
```

Later, Safaricom calls `POST /payments/callback` (a webhook, no JWT) and we mark the order `Paid`.

4. Running the project locally

```
cd /home/rome/x-uk-ecommerce-website
source venv/bin/activate          # turn on the Python environment
pip install -r requirements.txt   # install exact dependencies
uvicorn app.main:app --reload     # run the server
```

- `--reload` restarts the server when you save a file (great for development).
- Open **http://127.0.0.1:8000/docs** — FastAPI gives you a free interactive UI to test every endpoint. This is your best learning tool.
- Environment variables live in `.env` (never commit this file). Missing `SUPABASE_URL` / `SUPABASE_KEY` makes the app refuse to start on purpose.

Senior advice: Keep one terminal for the server, one for commands, one for logs. Read the error traceback bottom-up; the last line is the real problem.

5. The project tree

```
x-uk-ecommerce-website/
├── app/
│   ├── __init__.py
│   ├── main.py               # creates the app, CORS, registers routers
│   ├── database.py           # Supabase client (single shared instance)
│   ├── dependencies.py       # auth: get_current_user + require_admin (RBAC)
│   ├── services/             # reusable business logic
│   │   ├── __init__.py
│   │   ├── inventory.py      # restore stock, release stock (idempotent)
│   │   └── mpesa.py          # the ONE M-Pesa callback handler
│   ├── routers/              # one file per feature area (the "waiters")
│   │   ├── auth.py           # signup / login
│   │   ├── products.py       # catalogue CRUD
│   │   ├── orders.py         # checkout, tracking, admin order actions
│   │   ├── payments.py       # /payments/callback (canonical M-Pesa webhook)
│   │   ├── cart.py           # shopping cart
│   │   ├── wishlist.py       # saved items
│   │   ├── reviews.py        # product reviews (verified buyer)
│   │   ├── profiles.py       # user profile
│   │   ├── analytics.py      # admin dashboard numbers
│   │   ├── upload.py         # profile picture upload
│   │   ├── delivery.py       # delivery locations
│   │   ├── ai.py             # AI shopping assistant
│   │   └── daraja_service.py # low-level M-Pesa HTTP client (not a router!)
│   └── utils/
│       └── email.py          # receipt emails via Resend
├── supabase/
│   └── rls_policies.sql      # Row Level Security draft
├── tests/
│   └── simulate_mpesa_webhook.py  # fake a Safaricom callback
├── docs/
│   └── BACKEND_GUIDE.md      # this file
├── requirements.txt          # pinned dependencies
├── .env                      # secrets (git-ignored)
└── .gitignore
```

6. File-by-file walkthrough

For each file: **what it is**, **key code**, **what it does**, and **advice**.

`app/main.py` — the front door

```
app = FastAPI(title="GWV-xuk API")            # create the application object

_origins = [o.strip() for o in os.environ.get("CORS_ORIGINS", "").split(",") if o.st
if not _origins:
    _origins = ["http://localhost:3000", "http://localhost:5173"]  # dev defaults

app.add_middleware(                                  # allow browsers from these sit
    CORSMiddleware,
    allow_origins=_origins,
    allow_credentials=True,
    allow_methods=["*"],
    allow_headers=["*"],
)

app.include_router(products.router)   # plug each feature's routes into the app
app.include_router(orders.router)
# ... etc
```

- **What it does:** builds the FastAPI app, sets CORS, and mounts every router.

- **CORS explained:** by default a browser blocks JavaScript on `shop.com` from calling the API at `api.shop.com` unless the API says "I trust that origin". `CORSMiddleware` is that permission slip.
- **Advice:** in production set `CORS_ORIGINS="https://x-uk.com,https://admin.x-uk.com"`. Never use `allow_origins=["*"]` together with `allow_credentials=True` — browsers reject it and it's unsafe.

`app/database.py` — the shared pantry key

```
url = os.environ.get("SUPABASE_URL")
key = os.environ.get("SUPABASE_KEY")
if not url or not key:
    raise RuntimeError("Missing SUPABASE_URL or SUPABASE_KEY")
supabase: Client = create_client(url, key)
```

- **What it does:** creates **one** Supabase client reused everywhere.
- **Why one:** creating a new client per request wastes connections.
- **Advice:** import it, don't recreate it: `from app.database import supabase`.

`app/dependencies.py` — the bouncer + the VIP list (RBAC)

This is the heart of security. Two functions:

```
security = HTTPBearer()

async def get_current_user(credentials = Depends(security)):
    token = credentials.credentials              # the JWT string
    user_response = supabase.auth.get_user(token) # Supabase validates it
    return user_response.user                     # -> has .id and .email
```

- **What it does:** reads the `Authorization: Bearer <token>` header, asks Supabase "is this token genuine?", and returns the user. If not, `401`.
- **Analogy:** the bouncer checks your wristband is real.

```
_ADMIN_EMAILS = {e.strip().lower() for e in os.environ.get("ADMIN_EMAILS","").split(

async def require_admin(current_user = Depends(get_current_user)):
    if (current_user.email or "").lower() in _ADMIN_EMAILS:
        return current_user
    # otherwise look up profiles.role
    res = supabase.table("profiles").select("role").eq("user_id", current_user.id).s
    if isinstance(res.data, dict) and (res.data.get("role") or "").lower() == "admir
        return current_user
    raise HTTPException(403, "Admin privileges required")
```

- **What it does:** builds on the bouncer — "you're logged in, **and** you're on the VIP list". A normal customer gets `403 Forbidden`.
- **Two ways to be admin:** email in `ADMIN_EMAILS` env, **or** `profiles.role = 'admin'`. Pick one and stay consistent (env is fastest to set up; the DB column is better long-term).
- **How to use it in a route:**

```
@router.delete("/{product_id}")
async def delete_product(product_id: str, admin_user = Depends(require_admin)):
    ...
```

Senior advice: Put authorization in **one** place (`require_admin`) and reuse it. When each route rolls its own check, one route will eventually forget — and that's exactly the bug the old code had.

`app/services/inventory.py` — guarding the stock

```
STOCK_HELD_STATUSES = {"Pending PIN", "Processing"}

def restore_order_stock(order):
    for item in order.get("items", []):
        # add item["qty"] back onto products.stock_quantity
        ...

def release_stock_once(order, new_status="Payment Failed"):
    current_status = order.get("status")
    if current_status not in STOCK_HELD_STATUSES:
        return False                      # nothing to release
    res = (supabase.table("orders")
           .update({"status": new_status})
           .eq("id", order["id"])
           .eq("status", current_status)   # <-- ATOMIC guard
           .execute())
    if not res.data:
        return False                      # someone else already did it
    restore_order_stock(order)
    return True
```

- **Why it exists:** when stock is decremented at checkout, we must **give it back** if payment fails or is cancelled. Otherwise you slowly "lose" inventory.
- **The clever part:** `release_stock_once` uses `.eq("status", current_status)` in the WHERE clause. The database only updates the row *if it still has the old status*, so two duplicate webhook calls can't both add stock back.
- **Analogy:** two people trying to claim the last cinema seat — the database is the ticket machine; only the first press prints a ticket.

Beginner note: "idempotent" = doing it twice has the same effect as doing it once. Payment systems *must* be idempotent because webhooks get retried.

`app/services/mpesa.py` — the single payment brain

```
def process_stk_callback(payload):
    body = payload.get("Body", {}).get("stkCallback", {})
    checkout_request_id = body.get("CheckoutRequestID")
    result_code = body.get("ResultCode")

    order = ... # find order by checkout_request_id

    if result_code == 0:                    # success
        receipt = ... # pull MpesaReceiptNumber
        if order and order["status"] != "Paid":
            mark order Paid + save receipt
            send_receipt_email(...)
    else:                                    # failed / cancelled
        new_status = "Cancelled" if result_code == 1032 else "Payment Failed"
        release_stock_once(order, new_status)
```

- **What it does:** the **only** place M-Pesa results are interpreted. Both callback URLs call this function, so there's no duplicated logic to keep in sync (and no second bug to forget).

`app/routers/auth.py` — the signup / login desk

- `POST /auth/signup` — creates a user in Supabase.
- `POST /auth/login` — checks the password, returns `session` (JWT) + `user`.
- **Missing (build later):** logout, refresh token, forgot/reset password, email verification, and rate-limiting (to stop brute-force).
- **Advice:** never log passwords or tokens. Let Supabase handle password hashing — never roll your own.

`app/routers/products.py` — the catalogue

- `GET /products/` — list with **search, category, brand, color, min/max price, sort** (uses `.ilike`, `.eq`, `.gte`, `.lte`, `.order`).
- `POST /products/` — create (multipart form with images) — **ADMIN**.
- `PATCH /products/{id}` — partial update — **ADMIN**.
- `DELETE /products/{id}` — delete — **ADMIN**.
- `GET /products/{id}` — one product + its images (relation `product_images(image_url)`).

Code highlight — how filtering builds up:

```
query = supabase.table("products").select("*")
if search:  query = query.ilike("name", f"%{search}%")
if category: query = query.eq("category", category)
query = query.order(sort_field, desc=is_desc)
response = query.execute()
```

Each `if` only adds a filter when the client asked for it. This pattern is used a lot — learn it.

Note: the three write routes were previously open to any logged-in user. They now `Depends(require_admin)`. (The old `verify_admin` only checked "is logged in" — that was the RBAC bug.)

`app/routers/orders.py` — checkout, tracking, admin actions

Routes:

- `GET /orders/me` — my orders (token-scoped).
- `GET /orders/user/{user_id}` — **ADMIN** (was open to everyone = IDOR).
- `GET /orders/all` — **ADMIN**.
- `POST /orders/checkout` — the important one (below).
- `GET /orders/track/{order_ref}` — public live tracking by reference.
- `PATCH /orders/status/{order_id}` — **ADMIN**.
- `PATCH /orders/verify-payment/{order_id}` — **ADMIN** (manual receipt check).
- `POST /orders/status-result` — legacy alias that delegates to `services/mpesa.process_stk_callback`.

Checkout, explained (the security crown jewel):

```
for item in payload.items:
    product_row = supabase.table("products").select("name, price, stock_quantity")..
    if stock < item.qty: raise 400
    unit_price = float(product_row["price"])          # PRICE FROM DB, not client
    items_total += unit_price * item.qty

order_total = round(items_total + delivery_cost, 2)   # SERVER total
# 1) insert order  2) decrement stock (rollback on error)  3) STK push
```

- **The old bug:** it trusted `payload.total`. A user could send `total: 1` and buy an iPhone for KES 1. Now the total is **computed from database prices**.
- **Stock safety:** order is saved first, then stock decremented; if anything fails mid-way the stock is restored; if the STK push can't start, stock is released and the order marked `Payment Failed`.

`app/routers/payments.py` — the canonical callback

```
@router.post("/callback")
async def mpesa_callback(request: Request):
    payload = await request.json()
    process_stk_callback(payload)
    return {"ResultCode": 0, "ResultDesc": "Accepted"}
```

Matches `DARAJA_CALLBACK_URL` in `.env`. Both this and the legacy `/orders/status-result` share `services/mpesa.py`.

`app/routers/cart.py` — shopping cart

- `POST /cart/add` — add or increment (token-scoped).
- `GET /cart/` — list my cart with product details.
- **Missing:** update quantity, remove item, clear cart.

`app/routers/wishlist.py` — saved items

- `POST /wishlist/` and `GET /wishlist/` — both **token-scoped now**.
- **Old bug (IDOR):** accepted `user_id` in the body/URL, so anyone could read or add to someone else's wishlist. Now the id comes from the JWT only.

`app/routers/reviews.py` — reviews (verified buyer)

- `POST /reviews/` — only **token-scoped** now, and only if the order history contains that product. **Old bug:** trusted a client `user_id` (review spoofing) and compared the wrong field (`product_id` vs the stored `id`), so it always failed.
- `GET /reviews/{product_id}` — public list.

`app/routers/profiles.py` — user profile

- `GET /profiles/me` — my profile.
- `POST /profiles/me` — create/upsert my profile.
- `PATCH /profiles/me` — update my profile. **Old bug (IDOR):** `PATCH /profiles/update-profile/{user_id}` let anyone edit anyone's profile. Now the path is `/me` and the id comes from the token.

`app/routers/analytics.py` — admin numbers

- `GET /analytics/summary` — daily revenue, pending orders, low stock — **ADMIN** (was open before).
- **Missing:** sales over time, top products, conversion, per-location stats.

`app/routers/upload.py` — profile pictures

- `POST /upload/profile-picture` — uploads into `profile-assets/{my-user-id}/...`. **Old bug (IDOR):** took `{user_id}` from the URL; now it uses the token so you can only write into your own folder (matching the storage policy).

`app/routers/delivery.py` — delivery locations

- `GET /delivery/locations` — active locations + cost.
- **Missing:** admin CRUD for locations.

`app/routers/ai.py` — shopping assistant

- `POST /ai/chat` — token-scoped. Loads active products, builds a system prompt, calls OpenRouter, stores the conversation in `chat_history`.
- **Advice:** this is the blueprint for the WhatsApp triage bot — same idea, different channel. Later, extract the LLM-calling into `services/ai.py` so both reuse it.

`app/routers/daraja_service.py` — low-level M-Pesa client

- `get_access_token()` — OAuth token from Safaricom.
- `send_stk_push(...)` — triggers the payment prompt.
- `check_transaction(...)` — verifies a receipt.
- **Not a router** — it's a helper class, which is why it isn't registered in `main.py`. (A common confusion: things in `routers/` aren't automatically routes.)

`app/utils/email.py` — receipts

- `send_receipt_email(...)` — builds simple HTML and sends via Resend.
- **Advice:** wrap sends in try/except (already done) so a failed email never breaks a successful order.

`tests/simulate_mpesa_webhook.py` — fake Safaricom

- Sends a sample payload to your callback so you can test without real money.
- **Advice:** eventually replace with proper `pytest` tests using FastAPI's `TestClient` and mocked Supabase.

7. Security hardening — the 4 fixes explained

Fix 1 — RBAC (Role-Based Access Control)

- **Vulnerability:** the old `verify_admin` did `supabase.auth.get_user(token)` and returned success if the token was valid. It never checked *role*. Any logged-in customer could delete your inventory.

- **Fix:** `require_admin` in `dependencies.py` checks an admin email/role. Applied to product writes, all-orders, order status, verify-payment, analytics.
- **Lesson:** authentication ("who are you?") ≠ authorization ("what may you do?").

Fix 2 — IDOR (Insecure Direct Object Reference)

- **Vulnerability:** routes accepted `user_id` from the client, so a user could pass someone else's id and read/modify their data.
- **Fix:** the user id is taken **only** from the validated JWT (`current_user.id`). Affected: `profiles` , `upload` , `wishlist` , `reviews` , `orders/user/{id}` .
- **Lesson:** never let the client tell you who they are for *their own* data.

Fix 3 — CORS

- **Vulnerability:** no CORS config meant browsers would block the React app's requests in production.
- **Fix:** `CORSMiddleware` with origins from `CORS_ORIGINS` (dev defaults localhost:3000/5173).
- **Lesson:** CORS is a browser *permission*, not authentication. It does not protect the API from non-browser clients (curl/bots) — auth does.

Fix 4 — Checkout trust + stock consistency

- **Vulnerability:** the client sent `total` ; server trusted it. Tampered payload = pay almost nothing.
- **Fix:** server fetches each product's price, recomputes the total, validates stock/qty, persists the order, decrements stock with rollback, and releases stock exactly once on failed/cancelled payment. Duplicate M-Pesa callbacks merged and made idempotent.
- **Lesson:** money math and inventory are server-side, always.

8. How the M-Pesa payment flow works

```
Customer clicks "Pay with M-Pesa"
        |
        v
POST /orders/checkout  (server computes total from DB prices)
        |
        |-- 1. INSERT order (status "Pending PIN")
        |-- 2. UPDATE product stock (decrement)
        |-- 3. Daraja send_stk_push -> customer sees prompt on phone
        |
        v
Customer enters PIN on phone
        |
        v
Safaricom calls POST /payments/callback  (webhook, ResultCode 0 / 1032 / other)
        |
        v
services/mpesa.process_stk_callback
        |-- ResultCode 0  -> order "Paid" + receipt number + email
        |-- 1032          -> order "Cancelled" + stock restored
        |-- other         -> order "Payment Failed" + stock restored
```

- **STK push** = "SIM Toolkit" popup where the customer types their M-Pesa PIN.
- **Why a webhook?** The payment happens on the phone, asynchronously. We can't keep the HTTP request open for minutes. So Safaricom calls *us* when done.

- **Why stock restore matters:** if we decrement at checkout and the payment fails, that unit is "sold" in our eyes but never paid for.

9. Every endpoint that exists today

Method	Path	Auth	Purpose
POST	`/auth/signup`	public	Create account
POST	`/auth/login`	public	Get JWT session
GET	`/products/`	public	List/search/filter products
GET	`/products/{id}`	public	One product + images
POST	`/products/`	**admin**	Create product (images)
PATCH	`/products/{id}`	**admin**	Update product
DELETE	`/products/{id}`	**admin**	Delete product
GET	`/delivery/locations`	public	Active delivery locations
POST	`/orders/checkout`	public	Place order (guest checkout)
GET	`/orders/track/{ref}`	public	Track by reference
POST	`/orders/status-result`	public (webhook)	Legacy M-Pesa callback
POST	`/payments/callback`	public (webhook)	Canonical M-Pesa callback
GET	`/orders/me`	user	My orders
GET	`/orders/user/{id}`	**admin**	A user's orders
GET	`/orders/all`	**admin**	All orders
PATCH	`/orders/status/{id}`	**admin**	Update status
PATCH	`/orders/verify-payment/{id}`	**admin**	Verify M-Pesa manually
POST	`/cart/add`	user	Add/increment cart item
GET	`/cart/`	user	My cart
POST	`/wishlist/`	user	Add to wishlist
GET	`/wishlist/`	user	My wishlist
POST	`/reviews/`	user	Add review (verified buyer)
GET	`/reviews/{product_id}`	public	Product reviews
GET	`/profiles/me`	user	My profile
POST	`/profiles/me`	user	Create/update profile
PATCH	`/profiles/me`	user	Update profile
GET	`/analytics/summary`	**admin**	Dashboard summary
POST	`/upload/profile-picture`	user	Upload avatar

Method	Path	Auth	Purpose
POST	`/ai/chat`	user	AI assistant

10. Admin dashboard — endpoints to build

Suggested new prefix `app/routers/admin/`, mounted with `dependencies=[Depends(require_admin)]` so **every** admin route is protected by default (safer than remembering per-route).

Method	Path	Purpose
GET	`/admin/dashboard/stats`	Revenue, orders, customers, low stock
GET	`/admin/dashboard/sales?range=7d`	Sales time series
GET	`/admin/products?page=1`	Paginated product management
POST	`/admin/categories`	Create category
GET	`/admin/categories`	List categories
PATCH	`/admin/categories/{id}`	Update category
DELETE	`/admin/categories/{id}`	Delete category
POST	`/admin/products/{id}/images`	Add product image
DELETE	`/admin/products/{id}/images/{image_id}`	Remove image
GET	`/admin/orders?status=&page=`	Filter/paginate orders
POST	`/admin/orders/{id}/cancel`	Cancel + restore stock
POST	`/admin/orders/{id}/refund`	Refund record
GET	`/admin/customers`	List customers
GET	`/admin/customers/{id}`	Customer detail + orders
GET	`/admin/reviews`	Moderate reviews
DELETE	`/admin/reviews/{id}`	Remove review
POST	`/admin/coupons`	Create coupon
GET	`/admin/coupons`	List coupons
PATCH	`/admin/coupons/{id}`	Update coupon
DELETE	`/admin/coupons/{id}`	Delete coupon
GET	`/admin/delivery-locations`	Manage locations
POST	`/admin/delivery-locations`	Create location
PATCH	`/admin/delivery-locations/{id}`	Update location
DELETE	`/admin/delivery-locations/{id}`	Delete location
GET	`/admin/inventory/low-stock`	Restock alerts
PATCH	`/admin/inventory/{product_id}`	Adjust stock

11. Leads + WhatsApp triage bot — design & endpoints

Goal: a bot talks to customers on WhatsApp, qualifies them, and labels each as **hot / warm / cold**, handing hot leads to a human.

Data model (new tables)

```
leads(
  id uuid pk, phone text unique, name text, source text,
  interest text, budget numeric, location text, timeline text,
  status text default 'new',      -- new | warm | hot | cold | won | lost
  score int default 0,            -- 0..100
  assigned_to uuid,               -- team member
  last_message_at timestamptz, created_at timestamptz, updated_at timestamptz
)

whatsapp_messages(
  id uuid pk, lead_id uuid fk, direction text,  -- inbound | outbound
  body text, wa_message_id text, created_at timestamptz
)

lead_events(
  id uuid pk, lead_id uuid fk, type text, payload jsonb, created_at timestamptz
)
```

Flow

```
Customer messages WhatsApp business number
        |
        v
Meta calls POST /webhooks/whatsapp  (verify token first on GET)
        |
        v
services/whatsapp.handle_inbound
        |-- save inbound message
        |-- LLM (OpenRouter) replies, asking qualifying questions ONE at a time
        |-- LLM returns structured JSON: {intent, budget, urgency, score, stage}
        |-- update lead (status/score/last_message_at)
        |-- if score >= HOT_THRESHOLD -> notify human (WhatsApp to OWNER_WHATSAPP_NU
        v
Send reply back via WhatsApp Cloud API
```

Endpoints

Method	Path	Auth	Purpose
GET	`/webhooks/whatsapp`	public	Meta verification (hub.verify_token)
POST	`/webhooks/whatsapp`	public (signature-checked)	Receive messages
GET	`/admin/leads?status=hot&page=1`	admin	List/filter leads
GET	`/admin/leads/{id}`	admin	Lead detail
GET	`/admin/leads/{id}/messages`	admin	Conversation transcript
PATCH	`/admin/leads/{id}`	admin	Update status/assign/budget
POST	`/admin/leads/{id}/reply`	admin	Send manual WhatsApp reply
POST	`/admin/leads/{id}/assign`	admin	Assign to a team member
GET	`/admin/leads/stats`	admin	Funnel metrics

Scoring tips

- Ask **one** question per message (budget, device, timeline, location).
- Let the model output **structured JSON** so the backend can store a score.
- Define thresholds, e.g. `score >= 70 → hot`, `40–69 → warm`, `< 40 → cold`.
- Always verify Meta's request signature before trusting the webhook.

12. Recommended backend folder structure

Grow from today's flat `routers/` toward grouping by **audience**:

```
app/
├── main.py
├── core/
│   ├── config.py          # central settings (env parsing)
│   ├── security.py        # auth deps (moved from dependencies.py)
│   ├── logging.py         # structured logging
│   └── errors.py          # shared exception handlers
├── db/
│   ├── client.py          # Supabase client
│   └── migrations/        # SQL migrations
├── schemas/               # Pydantic models, one file per domain
│   ├── order.py
│   ├── product.py
│   └── lead.py
├── services/              # business logic (no HTTP here)
│   ├── inventory.py
│   ├── mpesa.py
│   ├── ai.py
│   ├── whatsapp.py
│   └── leads.py
└── routers/
    ├── public/            # customers/guests
    │   ├── products.py
    │   ├── orders.py
    │   ├── cart.py
    │   ├── auth.py
    │   ├── reviews.py
    │   └── ...
    ├── admin/             # mounted with dependencies=[Depends(require_admin)]
    │   ├── products.py
    │   ├── orders.py
    │   ├── customers.py
    │   ├── analytics.py
    │   └── leads.py
    └── webhooks/          # external callers
        ├── mpesa.py
        └── whatsapp.py
```

Why: at ~5 files the flat layout is fine. At ~30 files, grouping prevents "where is that route?" fatigue and lets you protect whole groups at once.

13. Files needed to complete the full project

Backend

- [x] `requirements.txt`
- [ ] `pyproject.toml` / pinned dev deps (`pytest` , `ruff` , `mypy`)
- [ ] `app/core/config.py` (centralised settings)
- [ ] `app/schemas/*` (extract Pydantic models out of routers)
- [ ] `app/routers/admin/*` (all admin endpoints above)
- [ ] `app/routers/webhooks/*` (move M-Pesa here; add WhatsApp)
- [ ] `app/services/whatsapp.py` , `app/services/leads.py` , `app/services/ai.py`
- [ ] `app/core/logging.py` , `app/core/errors.py`
- [ ] Auth extras: `/auth/logout` , `/auth/refresh` , `/auth/forgot-password` , `/auth/reset-password` , `/auth/verify-email`
- [ ] `coupons` , `addresses` , `categories` routers
- [ ] Pagination helper for all list endpoints
- [ ] `tests/` — unit + integration with `TestClient` , `conftest.py` , mocks

- [ ] `Dockerfile + docker-compose.yml`
- [ ] `.env.example` (documents every variable, no secrets)
- [ ] CI workflow (`.github/workflows/ci.yml`) running lint + tests

Database

- [ ] `supabase/migrations/*.sql` (reproducible schema)
- [x] `supabase/rls_policies.sql` (draft — review & run)
- [ ] Add `profiles.role` column; add `categories` , `coupons` , `addresses` , `leads` , `whatsapp_messages` , `lead_events`

Frontend (recommended)

```
frontend/
├── apps/
│   ├── web/        # React + Vite (or Next.js) storefront
│   ├── admin/      # React admin dashboard
│   └── mobile/     # Expo React Native (the APK)
└── packages/
    ├── api/        # typed API client generated from OpenAPI
    ├── types/      # shared TypeScript types
    └── ui/         # shared components
```

- React (web) and React Native (Expo) share language, patterns, and API calls — **not** DOM components. Share `api/` + `types/` in a monorepo (pnpm workspaces + Turborepo). For an APK, use Expo; a PWA is the no-store alternative.

14. Database & Row Level Security

- The app's own checks (RBAC, IDOR, checkout) protect users going **through the API**.
- **But** the Supabase anon key can be used to hit the database directly. **RLS** is the in-database lock: policies decide which rows each user can see/change.
- See `supabase/rls_policies.sql` for a ready-to-review draft. Key points:
  - `is_admin()` helper reads `profiles.role` .
  - Catalogue is public-read, admin-write.
  - `cart_items` , `wishlists` , `chat_history` , `profiles` are owner-only.
  - Orders: owner reads own, admins read/write all.
- **Rule of thumb:** the backend should use the **service_role** key (bypasses RLS) for its trusted operations, while the browser uses the **anon** key with RLS on. Never ship the service_role key to the frontend.

15. Prioritised next steps

1. **Turn on RLS** using `supabase/rls_policies.sql` , add `profiles.role` .
2. **Add `CORS_ORIGINS` and `ADMIN_EMAILS` to `.env`** (see `.env.example`).
3. **Pin dev tooling:** `ruff` (lint), `pytest` (tests), `mypy` (types).
4. **Write DB migrations** so the schema is reproducible.
5. **Make Supabase calls async** (or wrap in `run_in_threadpool`) so the event loop isn't blocked under load.

6. **Complete auth flows** (logout/refresh/reset) + rate-limit login.
7. **Build admin routers** (section 10) behind `require_admin`.
8. **Build the WhatsApp bot** (section 11) with leads tables and scoring.
9. **Add feature gaps:** categories, coupons, addresses, cart update/remove, order cancel/refund, pagination.
10. **Observability:** structured logging, Sentry, health check `/health`.
11. **Then build the frontend** (web + admin + Expo) against this API.
12. **Tests + CI** so changes don't break the security fixes.

Senior advice: do the boring foundations (migrations, tests, logging) *before* adding many features. They feel slow up front and save weeks later. And always write a test for every security bug you fix, so it can never come back.

16. Glossary for juniors

- **API** — the set of URLs your frontend calls.
- **Endpoint** — one URL + HTTP method (e.g. `POST /orders/checkout`).
- **Router** — a group of related endpoints in one file.
- **Dependency** — a function FastAPI runs before your route (e.g. auth check).
- **Middleware** — code that runs for every request (e.g. CORS).
- **DTO / Schema** — a Pydantic class describing request/response shape.
- **JWT** — signed token proving who you are.
- **RBAC** — role-based access control (admin vs customer).
- **IDOR** — using someone else's id to access their data (a classic bug).
- **Idempotent** — safe to run more than once.
- **Webhook** — an external service calling your URL with an event.
- **RLS** — database rules deciding which rows a user may access.
- **Migration** — a versioned file that changes the database schema.
- **Service layer** — reusable business logic, separate from HTTP routes.

End of guide. Keep this file next to your code and update it as you build. A guide that lies is worse than no guide.

- Thought: 574ms All done. Here's what I created: Files added File requirements.txt supabase/rls_policies.sql .env.example docs/BACKEND_GUIDE.md What's in docs/BACKEND_GUIDE.md 16 sections, written for a junior dev:

1. The big picture — restaurant analogy (frontend = dining room, API = waiter, auth = bouncer, services = kitchen, Supabase = pantry, M-Pesa = cashier, webhooks = messenger)
2. Stack explained from scratch (FastAPI, Pydantic, JWT, webhook, etc.)
3. A request's full journey through the app (diagram)
4. How to run it locally
5. The project tree
6. File-by-file walkthrough — every file with code snippets, what it does, and senior advice
7. The 4 security fixes explained (the vuln → the fix → the lesson, for each)
8. The M-Pesa payment flow step-by-step
9. Every endpoint that exists today (table)
10. Admin dashboard endpoints to build (full table)

11. Leads + WhatsApp triage bot — data model, flow, endpoints, scoring tips
12. Recommended backend folder structure (current → target)
13. Files needed to complete the full project (backend / DB / frontend checklists)
14. Database & RLS explained
15. Prioritised next steps
16. Glossary for juniors Verified: import app.main still succeeds after all changes. To read it: open docs/BACKEND_GUIDE.md, or from your editor it renders as markdown. Suggested order for learning: sections 1–3 first (mental model), then 6 (files), then 7–8 (security/ payments).